

BronchoTrack_V3 — Progress Update

A plain-language walkthrough of where things stand and what's changed

From: Aveesha Nishendra

Date: September 2, 2026

Hi — wanted to put together a proper update on the bronchoscope tracking project, since it's been a while since the last check-in and there's a genuine story to tell here: something didn't work, I figured out why, tried a fix that turned out to be worse, and then landed on one that actually helped. I've tried to keep this readable rather than dense — happy to go deeper on any part of it if it'd help, and I've got a much more technical version of all of this ready to send too if you want the full detail at any point.

Quick recap of the bigger picture, since it's been a minute: the end goal of this whole thread of work is a system that can watch bronchoscope video in real time and tell a doctor which airway branch the scope is currently sitting in, using nothing but the video itself plus a 3D model built from the patient's own CT scan ahead of time. No extra tracking hardware, no manual annotation during the procedure. This update covers a full development cycle on that system: a real bug I found, a first attempt at fixing it that I ended up abandoning, and the fix that actually stuck.

What the system actually does

The goal is to look at live bronchoscope video and figure out, automatically, which airway branch the camera is currently sitting in — trachea, left main bronchus, one of its sub-branches, and so on. This is useful because it means a scope could eventually tell a doctor "you're here" on a map of the patient's own airways, without needing a separate tracking hardware setup.

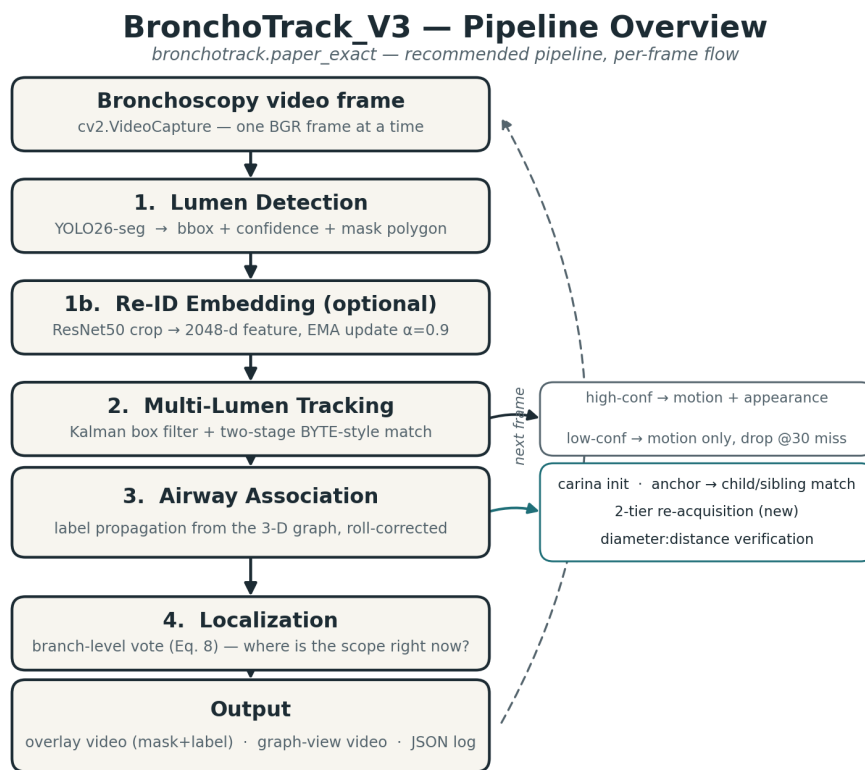
It works by combining two things. First, a pre-operative 3D model of the patient's airway tree, built from their CT scan — this gives us the known shape and branching pattern ahead of time. Second, the live video: the software detects the dark, roughly circular openings in the airway wall (each one is the entrance to a branch), tracks them frame to frame as the camera moves, and matches what it's currently seeing against what the 3D model says should be visible from wherever the scope currently is. When those two agree, it commits to a label — "this opening is the right upper lobe branch" — and that becomes the basis for reporting the scope's location.

This is based on a published method (a paper called BronchoTrack), and I've been building a careful, faithful implementation of it, plus a few small, clearly-flagged improvements on top where the paper itself is vague or where I found something in practice that needed fixing. I've kept those two things separate on purpose — there's a version of the code that sticks strictly to what the paper describes, so I always have a clean baseline to compare any of my own additions against, and can honestly say which results come from the paper's own method versus something I added.

It's worth saying plainly why this matters beyond just "it's a cool tracking demo": right now, bronchoscopists largely navigate by memory and visual landmarks, and getting lost or disoriented deep in a small, branching airway is a real, recognized problem, especially for less experienced operators or unusually complex anatomy. A reliable, camera-only localization system is a step toward software that could eventually assist with that in real time, without needing any additional imaging hardware in the room.

How a frame moves through the system

Roughly, for every single frame of video: the model detects the airway openings visible in that frame; a tracker keeps each opening's identity consistent from one frame to the next (so "opening #3" stays "opening #3" even as the camera moves and it drifts around the screen); the airway-matching step tries to figure out which real anatomical branch each currently-visible opening corresponds to, using the 3D model as a reference; and finally, once openings are labeled, the system takes a vote among everything currently visible to decide the scope's overall location. The diagram below shows that same flow.



How one video frame flows through the pipeline, start to finish.

A couple of things about this that I think are worth calling out, because they weren't obvious to me going in. First, the detector doesn't just draw a box around each opening — it segments the actual shape, pixel by pixel, which turns out to matter a lot: the size and shape of an opening (not just its rough position) is itself useful evidence for which branch it is, and a plain box would throw that information away. Second, there's a deliberate, independent double-check built in: even after an opening gets matched to a branch by position, the system separately checks whether that opening's apparent size, at that distance, actually looks like

what the 3D model predicts it should look like from there. A label only gets treated as fully confirmed once both checks agree — which is stricter than it sounds, and comes up again later in this update.

The problem I found

The system was already working reasonably well, but it had a real weak spot: it only ever labels a new opening by comparing it against an opening it's already confident about — an "anchor". That's sensible most of the time. But if the video has a rough patch — motion blur, the scope losing sight of everything for a moment, a burst of frames where the detector just doesn't pick anything up — and every single anchor gets lost at the same time, the system had no way to recover. It would just stop reporting a location for the rest of the video, because it no longer had anything to compare new detections against.

This wasn't a hypothetical concern — I confirmed it on a real patient recording. Out of a roughly 1,000-frame video, the system lost its last anchor around a third of the way through, and simply went blank for the remaining 80%+ of the video. That's obviously not good enough for something meant to track a scope's position through an entire procedure.

First attempt: a bigger redesign that didn't pan out

My first instinct was to solve this more thoroughly, so I built a fairly involved alternative: instead of relying on a persistent "anchor" at all, it would combine three separate signals every single frame — how fast a visible opening was growing in the frame (a cue for "the scope is heading toward this one"), a position-based match against the 3D model, and a shape-based match that used the entire detected outline of each opening rather than just its center point — and blend all three together with a smoothing filter into one running confidence score per branch. The appeal was that it would never be dependent on any single tracked object surviving from one frame to the next, so the dead-end problem above couldn't happen by construction.

I built the whole thing, tested it carefully, and ran it on the same real video. But when I compared it side by side against the original, simpler system, it was genuinely worse — more complexity without a matching improvement in reliability. So I made the call to set it aside rather than push it further, and went back to the version that was already working, to fix the actual problem in a smaller, more targeted way instead.

I'm including this attempt in the update rather than quietly skipping past it, because I think it's a genuinely useful data point, not just wasted time: it confirmed that the existing anchor-based approach was fundamentally sound and just needed a bounded patch for one specific failure mode, rather than a ground-up rethink. That's a real conclusion, even though the code itself didn't end up shipping as the main path. It's still sitting in the codebase, tested and working, in case it's ever useful as a reference or a starting point for something else down the line.

What actually fixed it

The fix I landed on keeps the original "anchor" approach, but gives it a memory. The system now keeps a snapshot of the last opening it was confident about, even after that opening disappears from view. If every anchor is lost, instead of giving up, it tries two things:

- First, it checks whether one of the still-unidentified openings in the current frame is probably just the same opening reappearing — the scope's own current position, coming back after a rough patch — by seeing how closely it overlaps with where that opening was last seen. If so, it just picks up the same label directly, no need to re-derive anything from scratch.
- If nothing matches closely enough for that, it falls back to using the last known position as a stand-in reference point, so newly-visible branches nearby can still be identified against the 3D model, even though the original opening itself is gone.

Either way, the moment something gets re-identified, the system is back to normal and keeps working from there — this isn't a one-time reset, it can kick in again and again through a long video. And there's a safety cutoff: if too much time has passed since anything was last confidently seen (about 3 seconds), the system stops trusting that old snapshot and waits for a fresh one instead, rather than guessing off something that's gone stale.

Did it actually help?

I re-ran the fix on the exact same real video that showed the original blank-screen problem, comparing before and after under otherwise identical settings:

	Before	After
Frames with a location reported	about 15%	about 40%

Same 1,000-frame video, same settings, before vs. after the fix.

So, roughly two and a half times as much of the video now gets a location at all — a real, meaningful improvement, and it confirms the system no longer permanently freezes after that kind of gap.

One honest caveat, though: there's a separate, stricter check in the system that decides whether a label is confident enough to actually be drawn on the video (it double-checks the label against the 3D model's expected geometry, not just against the tracking). That stricter number didn't move much — it stayed at roughly 15% either way. That's not a contradiction; it just means the fix is doing exactly what it's supposed to (recovering a label after a gap), while that separate, independent confidence check is unaffected by it and is really a different question about how strictly the system should trust a given match. I think that's worth a conversation about whether that threshold should be relaxed a little for a label that came back this way, versus one that was matched fresh.

A smaller visual improvement too

Separately, I also improved how the output video looks. It used to just draw a small dot at the center of each identified opening. Now it draws the actual detected shape — a filled, semi-transparent outline of the real opening — which makes it much easier to visually sanity-check what the system is picking up, rather than trusting a single point. One thing this surfaced: occasionally an opening near the edge of the frame gets detected with a slightly odd, oversized shape, because the detection model's outline runs along the frame border rather than stopping cleanly at the opening's real edge. It's a minor visual quirk in the raw detections, not a tracking bug, but I flagged it as something worth a filter if it turns out to be distracting.

How confident I am in this

Everything above is backed by automated tests — 58 in total across the different parts of the system — that I run fresh every time before calling something done, including from a completely clean copy of the code (not just my working folder) to make sure nothing about my own environment is hiding a problem. For the recovery fix specifically, that includes tests for the normal case (losing a lumen briefly and getting it back), the edge case where too much time has passed and the system correctly gives up rather than guessing, and an on/off switch that reproduces the original behavior exactly, so I can always fall back to a clean before/after comparison.

Beyond the automated tests, I made a point of not just trusting the summary numbers. I went back and visually checked the actual output video frame by frame in a few places — early on, in the middle, and later in the recording — which is exactly how I caught the detail about the stricter confidence check staying flat. It would've been easy to just report the 15%-to-40% jump and call it a clean win; I'd rather flag the nuance than oversell it.

Where things stand, and what's next

The recommended, actively-developed version of the system now includes this recovery fix and the improved video output, and it's the one I'd point to as current. The earlier three-signal redesign still exists in the codebase in case it's ever useful as a reference, but it's not the direction I'm continuing with.

A couple of open questions I'd like your take on: whether that stricter confidence check should treat a "recovered" label a bit more leniently than a freshly-matched one, and whether it's worth spending time cleaning up the frame-edge detection quirk mentioned above before the next round of testing.

I've also got the fuller technical writeup (architecture details, the exact matching logic, full test breakdown) and the demo videos ready to send if useful — just let me know.

Thanks,
Aveesha